
Assignment 1: Bare Metal Raspberry Pi Setup (10pt)

Deadline: October 11th, 2026; 23:59 CET

In this assignment, you will set up your local programming environment for “bare metal” development on the Raspberry Pi platform and create, compile, and run a few simple programs on the hardware. To solve the assignment, you need a *Raspberry Pi 400 PC-Kit*, a *330Ω resistor*, an *LED*, a *Breadboard*, and two *Male-to-Female jumper wires*.

Everything you need is in the assignment repository:

<https://github.com/Interactions-HSG/2026-HS-ICS-Assignment1>

Start by reading the `README.md` there – it walks you through creating your own copy, step by step, and assumes you have never used Git before.

By the deadline, you hand in a **GitHub repository**. There is no zip file: on Canvas you submit **only the URL of your repository**. Your repository must contain:

- **Tasks 1, 2, 3 and 5:** Your answers in the file `REPORT.md`. All questions below are already written out in that file – you only fill in the blanks. Please also state there the names of all team members and their roles, the major pitfalls you encountered, and how many minutes/hours you spent on this assignment.
- **Task 2:** A screenshot named `screenshots/t1.png` of your console window showing the version of your `arm-none-eabi-objcopy` command.
- **Task 4:** Your code in the file `led_blink.c`
- **Bonus Task:** Your code in the file `led_fade.c`

Two things we cannot grade without, so please do not forget them:

- Your repository must be **private** – use the **Use this template** button on the page above, **do not fork**.
- You must add **lukabekavac** and **Karimkh31** as **collaborators** on your repository.

We wish you a lot of fun with your first (?) embedded computing project!

On using AI: you are encouraged to use AI tools for this assignment. But do not ask “solve this for me” – ask “explain how to solve this”. The point of this assignment is that you end up able to read a hardware manual and reason about registers yourself. Note down in `REPORT.md` which tools you used and what for.

This assignment generally follows a tutorial on [valvers.com](https://www.valvers.com)¹, however with a few twists:

¹<https://www.valvers.com/open-software/raspberry-pi/bare-metal-programming-in-c-part-1/>

- You do **not** need to work on the *Generating SD Cards* part of the tutorial, as we provide the relevant files for your SD card to enable you to focus on the essence of this assignment.
- For the same reason, the repository contains a file `power_blink.c` that is derived from the `armc-03.c` file in the tutorial – this file is specific to our version of the Raspberry Pi and thus simpler and easier to understand.
- The tutorial does not solve the assignment for you – it merely presents you with a solution to a *similar* problem that you will need to adapt to fit your needs.
- See `README.md` in the repository to get started, and `docs/TOOLCHAIN.md` for the exact commands you will need.

① Bandit Game: level 3 - 10 (1pt)

In this task, you are asked to work through the **OverTheWire Bandit**² challenges from *Level 3* -> *Level 4* up to *Level 9* -> *Level 10*.

You are asked to document both the *steps* and the *passwords* you found. For each level, write down:

- The command(s) you used
- The password you discovered
- How you found the solution

One-Liner Challenge: As an additional challenge, you are encouraged to combine multiple commands into a single line using pipes (|) and command chaining (&&, ;) wherever possible. If a solution requires multiple commands, try to chain them into one efficient command.

② Toolchain Setup (1pt)

In this task, you set up and test your toolchain for compiling bare metal programs and deploying them onto your Raspberry Pi.

Setting up your Programming & Compilation Environment

Set up a GCC ARM compilation environment on your machine (available for Linux, Windows and Mac from here³), where we will be working with the `arm-none-eabi-xx` toolchain for 32bit systems (i.e. *ARM processor / NO operating system / embedded Application Binary Interface*). Install this toolchain (version 13.3.Rel1);

- For Windows: Download and use `arm-gnu-toolchain-13.3.rel1-mingw-w64-i686-arm-none-eabi.exe`. You can add the installation directory to the `PATH` environment variable at the end of the installation wizard (See Figure 1).
- For Mac: Download and use `arm-gnu-toolchain-13.3.rel1-darwin-x86_64-arm-none-eabi.pkg` or `arm-gnu-toolchain-13.3.rel1-darwin-arm64-arm-none-eabi.pkg` for Apple silicon (You can check the processor in system information).

Add the path to the toolchain bin directory to the environment variable `$PATH`

On MacOS, the executable binaries are installed at `/Applications/ArmGNUToolchain/{your toolchain version}/arm-none-eabi/bin`. To add this to your `PATH` environment variable, use the command given in `docs/TOOLCHAIN.md`. This command appends a single line at the end of your ZSH (the default shell on MacOS) user configuration file.

If the toolchain is successfully installed, you should see something like Figure 2 when typing `arm-none-eabi-gcc --version[ENTER]` in Mac Terminal or `arm-none-eabi-gcc-13.3.1.exe --version` in Windows PowerShell; also see `docs/TOOLCHAIN.md` for this.

③ Bare Metal “Hello World” (4pt)

In this task, you compile and deploy a simple program that will blink your Raspberry Pi’s power LED – a classic “Hello World” program for embedded computing.

(a) Prepare the SD card (0.5pt)

²<https://overthewire.org/wargames/bandit/bandit4.html>

³<https://developer.arm.com/downloads/-/arm-gnu-toolchain-downloads>

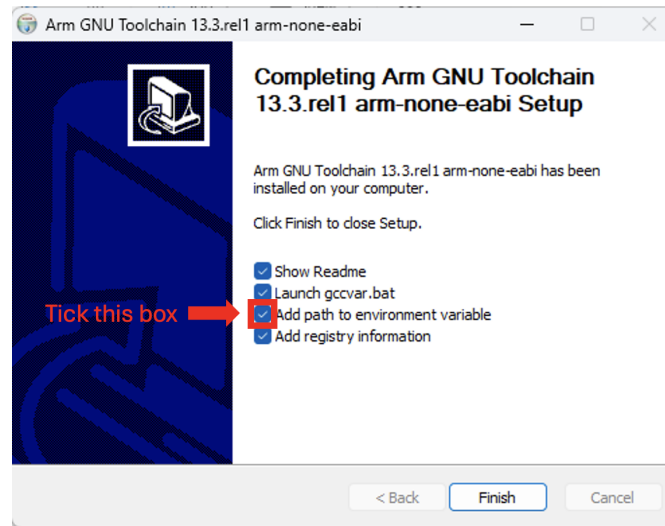


Figure 1: The GNU Toolchain installation wizard.

```

jingwu — -zsh — 76x14
~ — -zsh
(base) jingwu@eduroamverw-130-82-31-227 ~ % arm-none-eabi-gcc --version
arm-none-eabi-gcc (Arm GNU Toolchain 13.3.Rel1 (Build arm-13.24)) 13.3.1 202
40614
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

(base) jingwu@eduroamverw-130-82-31-227 ~ %

```

(a) In MacOS Terminal

```

PS C:\Users\ > arm-none-eabi-gcc-13.3.1.exe --version
arm-none-eabi-gcc-13.3.1.exe (Arm GNU Toolchain 13.3.Rel1 (Build arm-13.24)) 13.3.1 20240614
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

```

(b) In Windows PowerShell

Figure 2: Show the installed version of arm-none-eabi-gcc.

Format the SD card in **FAT32** file system⁴ so that the RPi can use your custom kernel image to boot up with. The SD card also needs to contain specific files. Copy the 3 files (config.txt, fixup4.dat, and start4.elf) that you find in the boot/ directory in your repository onto the formatted SD card – do not create a boot/ directory on the SD card, just copy the files.

Raspberry Pi 400 uses an EEPROM to boot the system⁵. The bootloader can be configured using the file config.txt⁶ and therefore a minimal firmware is provided in fixup4.dat and start4.elf. For the Raspberry Pi 400, the default filename on the boot partition to use when loading the kernel is kernel71.img⁷.

(b) **Blink the power LED (0.5pt)**

Using the toolchain from Task 2, “cross-compile” the code to be executed on the Raspberry Pi

⁴Find instructions online.

⁵<https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#raspberry-pi-4-boot-eeeprom>

⁶https://www.raspberrypi.com/documentation/computers/config_txt.html#boot-options

⁷https://www.raspberrypi.com/documentation/computers/config_txt.html#kernel

400's Broadcom chip, the BCM2711⁸. Follow the instructions in `docs/TOOLCHAIN.md` to produce a `kernel71.img` file (or simply run `make power_blink`) and copy this binary kernel image to the SD card. Eject the SD card from your computer, insert it into the Raspberry Pi, and connect the power adapter. The Raspberry Pi's Power LED should be blinking now.

(c) **Examine the code and understand the BCM2711 (3pt)**

Examine the code in `power_blink.c`, together with the manual of the BCM2711 ARM Peripherals⁹ – particularly interesting for this assignment are pages 65-70 of the manual, as well as pages 4-6. Answer the following questions in `REPORT.md`:

- 1.) In line 51 of `power_blink.c`, what is the register address represented by `gpio[GPFSEL4]` according to the manual? Provide your answer as an offset from the GPIO base address.
- 2.) Calculate the value `(1 << 6)` and answer in binary notation. According to the manual, what does it mean when this value is written to the address `gpio[GPFSEL4]`?
- 3.) How would you modify this line in order to turn all the GPIO pins that are covered by the **GPIO Alternate Function Select Register 1** (i.e., GPIO 10-19) into **outputs**? (HINT: Use 0 for the reserved bit values.)

At this point, you should understand what this program does and why it works.

④ **Building your own Digital Circuit and Blinking an LED (4pt)**

Use your cables, resistors and breadboard to properly connect an LED to your Raspberry Pi 400 at **GPIO 16**¹⁰ (= The Pin 36) as shown in Figure 3.

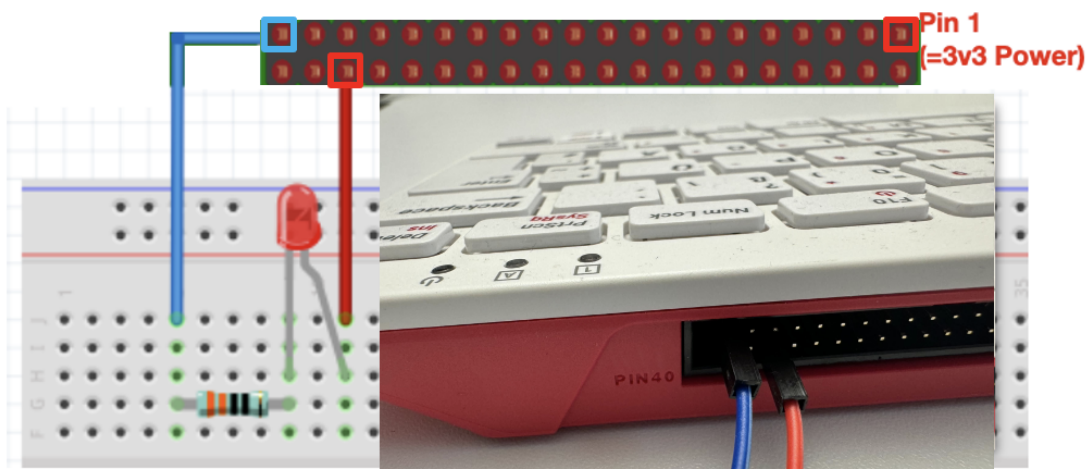


Figure 3: The wiring diagram.

Your repository already contains `led_blink.c`, which starts out as a copy of `power_blink.c`. Modify it so that it blinks the *external* LED that you connected to GPIO 16 instead of the Raspberry Pi's power LED. Build it with `make led_blink`. To solve this puzzle, you will need to consider again the BCM2711 manual.

⑤ **BONUS: LED Experiments (2pt)**

⁸<https://www.raspberrypi.com/documentation/computers/processors.html#bcm2711>

⁹<https://datasheets.raspberrypi.com/bcm2711/bcm2711-peripherals.pdf> (also provided in `docs/` in the repository)

¹⁰<https://pinout.xyz/>

Extend your program in the file `led_fade.c` (also already in your repository) with functionality that fades the LED in and out slowly instead of just turning it on/off. There are multiple options of how this can be accomplished – any functionally correct way will be accepted as a solution to this task!